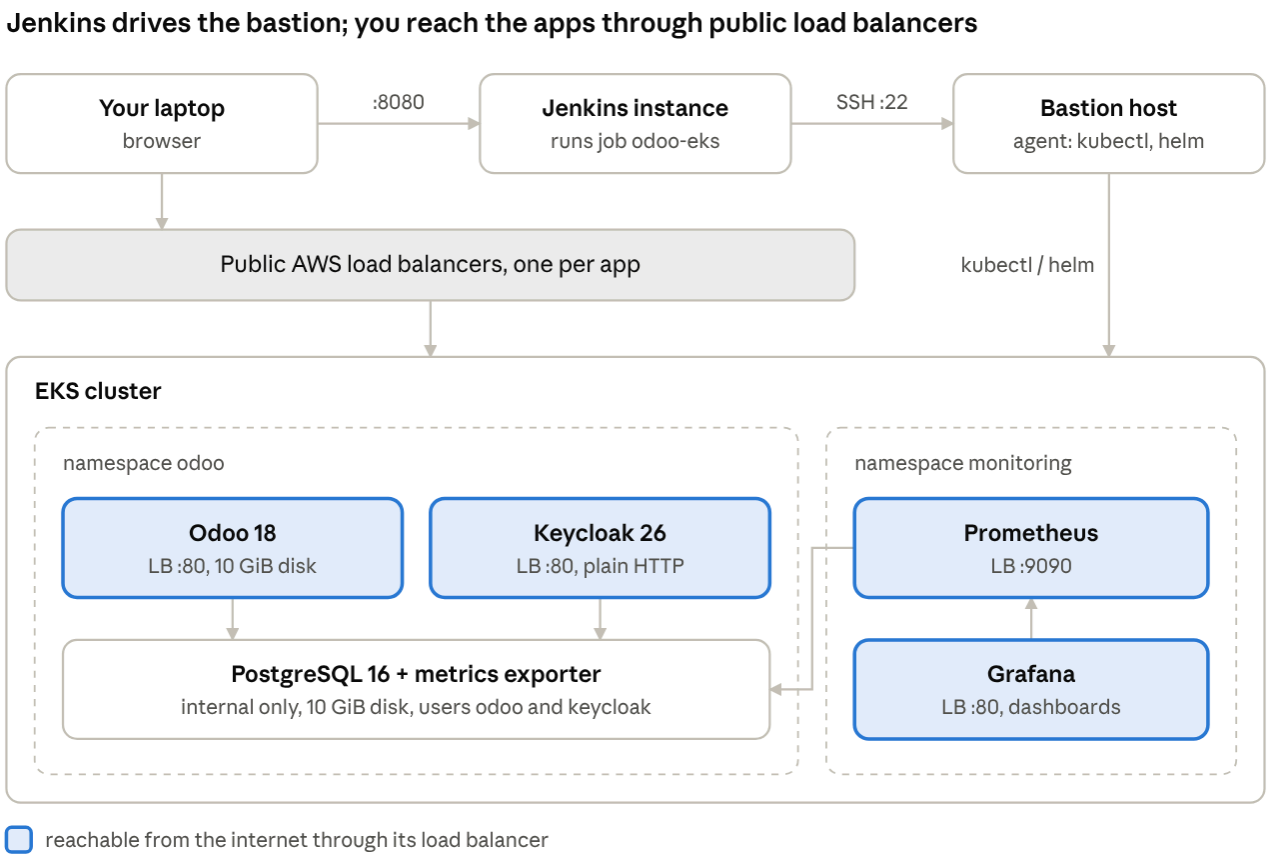
Odoo 18 on EKS – Runbook

Oct 13, 2026 · @Kiran

Overview

One Jenkins job deploys Odoo 18, PostgreSQL 16, Keycloak 26, Prometheus and Grafana to the EKS cluster `odoo-eks-cluster` (us-east-1). Jenkins runs on its own instance and executes the job on the bastion host over SSH. Each app gets its own public AWS load balancer, so there are no domains, no ingress and no TLS.



architecture · Jenkins, bastion, EKS apps

Odoo and Keycloak store data in the shared PostgreSQL; Prometheus scrapes its exporter on port 9187 and Grafana reads from Prometheus.

App	URL	Login
Odoo 18	`http://<odoo-lb-address>`	Create a database on first visit with the Odoo master password

App	URL	Login
Keycloak 26	`http://<keycloak-lb-address>/admin`	`admin` / keycloak-admin-password
Grafana	`http://<grafana-lb-address>`	`admin` / grafana-admin-password
Prometheus	`http://<prometheus-lb-address>:9090`	none
Jenkins	`http://<jenkins-instance-ip>:8080`	your Jenkins admin user

The last stage of every deploy prints the real load balancer addresses. PostgreSQL is internal only: Odoo connects as user `odoo`, Keycloak as user `keycloak` with its own database.

Repository: `https://github.com/kiran249/project-odoo18`, branch `claude/eloquent-rubin-7080hm`.

File	What it does
`Jenkinsfile`	The pipeline: connect to EKS, secrets, Postgres, Odoo, Keycloak, monitoring, print URLs; or destroy
`k8s/storageclass.yaml`	`ebs-gp3` storage class for EBS disks
`k8s/postgres.yaml`	PostgreSQL StatefulSet, 10 GiB disk, metrics exporter sidecar on port 9187
`k8s/odoo.yaml`	Odoo Deployment, 10 GiB disk, public load balancer on port 80
`k8s/keycloak.yaml`	Keycloak Deployment on plain HTTP, public load balancer on port 80
`monitoring/values.yaml`	kube-prometheus-stack settings: public load balancers, Postgres scrape job, PostgreSQL dashboard
`scripts/setup-bastion.sh`	One-time install of Java, git, AWS CLI, kubectl, Helm (and Jenkins) on Amazon Linux

1. AWS prerequisites (one time)

Four things must be in place in AWS before the first deploy: working EBS storage, bastion access to the cluster, tagged public subnets and open ports.

1.1 EBS storage (Postgres and Odoo disks)

1. **IAM → Roles → Create role**: trusted entity **AWS service → EKS → EKS - Pod Identity**, permission `AmazonEBSCSIDriverPolicy`, name `ebs-csi-role`.
2. **EKS → odoo-eks-cluster → Add-ons**: install **Amazon EKS Pod Identity Agent** if it is missing.
3. **Add-ons → Get more add-ons → Amazon EBS CSI Driver**: install it, and under **Pod Identity IAM role** pick `ebs-csi-role`.
4. Check on the bastion: `kubectl get pods -n kube-system | grep ebs-csi-controller` must show `6/6 Running`.

1.2 Bastion access to the cluster

1. Give the bastion an IAM role: **EC2 → bastion → Actions → Security → Modify IAM role**. The role needs `eks:DescribeCluster` (for example `AmazonEKSClusterPolicy`).
2. **EKS → odoo-eks-cluster → Access**: authentication mode must include **EKS API**.
3. **Create access entry**: principal = the bastion role ARN (or the IAM user whose keys are in `~/.aws`), policy `AmazonEKSClusterAdminPolicy`, scope **Cluster**.

1.3 Public subnets for the load balancers

The worker nodes have private IPs only, so apps are reached through load balancers. Tag every **public** subnet of the cluster VPC (route table → Internet Gateway): **VPC → Subnets → Tags →** `kubernetes.io/role/elb` = `1`.

1.4 Security groups

Instance	Inbound rule	Source
Jenkins instance	TCP 8080	your IP
Bastion	TCP 22	Jenkins instance (its private IP or security group)

The load balancers create their own security group rules for ports 80 and 9090.

2. Bastion host (one time)

The bastion runs Amazon Linux and needs Java, git, AWS CLI v2, kubectl and Helm. The pipeline runs there as `ec2-user`.

1. Clone the repo and run the setup script (it re-runs itself with `sudo` if needed):

```
sudo dnf install -y git
git clone -b claude/eloquent-rubin-7080hm
https://github.com/kiran249/project-odoo18.git
cd project-odoo18
sudo bash scripts/setup-bastion.sh
```

2. The script also installs Jenkins. Jenkins runs on its own instance, so stop the bastion copy:

```
sudo systemctl disable --now jenkins
```

3. Create the Jenkins agent folders:

```
mkdir -p /home/ec2-user/jenkins/tmp
```

4. Verify cluster access as `ec2-user`. All three must succeed before any pipeline run:

```
aws sts get-caller-identity
aws eks update-kubeconfig --region us-east-1 --name odoo-eks-cluster
kubectl get nodes
```

3. Jenkins (one time)

Jenkins needs six credentials, the bastion as an SSH agent labelled `bastion`, and one pipeline job.

3.1 Credentials

Manage Jenkins → Credentials → System → Global credentials → Add Credentials. Use long passwords without `'` or `$`. A short password that also appears in the cluster name shows up as `****` in logs.

ID	Kind	Value
`db-password`	Secret text	PostgreSQL password (Odoo and Keycloak users)

ID	Kind	Value
`odoo-master-password`	Secret text	Odoo master password, used to create databases
`keycloak-admin-password`	Secret text	Keycloak `admin` password
`grafana-admin-password`	Secret text	Grafana `admin` password
`github`	Username with password	GitHub username + Personal Access Token
`bastion-ssh`	SSH Username with private key	Username `ec2-user`, the bastion `.pem` key pasted in

3.2 Bastion as an agent

1. **Manage Jenkins → Plugins → Available**: install **SSH Build Agents**.
2. **Manage Jenkins → Nodes → New Node**: name `bastion`, **Permanent Agent**.
3. Settings:
   - Remote root directory: `/home/ec2-user/jenkins`
   - Labels: `bastion`
   - Launch method: **Launch agents via SSH**, Host = bastion private IP, Credentials = `bastion-ssh`, Host Key Verification Strategy = **Non verifying**
   - Launch method → Advanced → JVM Options: `-Djava.io.tmpdir=/home/ec2-user/jenkins/tmp`
4. Save, then check the node shows **online**.

3.3 Node monitor thresholds

Amazon Linux keeps `/tmp` in memory (about 456 MiB), and Jenkins takes agents offline below 1 GiB free. **Manage Jenkins → Nodes → gear icon (Node Monitors)**: set **Free Temp Space** threshold to `100MiB` (warning `200MiB`), or tick **Don't mark agents temporarily offline**. Do the same for **Free Swap Space**.

3.4 Pipeline job

1. **New Item** → name `odoo-eks` → **Pipeline**.
2. Pipeline → Definition **Pipeline script from SCM**, SCM **Git**:
   - Repository URL: `https://github.com/kiran249/project-odoo18.git`

- Credentials: `github`
- Branch Specifier: `*/claude/eloquent-rubin-7080hm`
- Script Path: `Jenkinsfile`

3. Save. The first **Build Now** only loads the parameters and may fail; that is expected.

4. Deploy, first login, destroy

A deploy takes 10–15 minutes and is safe to re-run: it updates whatever already exists.

4.1 Deploy

odoo-eks → Build with Parameters:

Parameter	Value
`ACTION`	`deploy`
`CLUSTER_NAME`	`odoo-eks-cluster`
`AWS_REGION`	`us-east-1`

The console should say `Running on bastion`. Stages run in this order: Connect to EKS → Namespaces & Secrets → PostgreSQL → Odoo → Keycloak → Prometheus & Grafana → Access URLs. The last stage prints the four load balancer addresses; new load balancers need 2–3 minutes before they answer.

4.2 First login to each app

App	First steps
Odoo	Enter the master password, a database name, admin email and password → **Create database**
Keycloak	Open `/admin`, log in as `admin`. The pipeline sets the `master` realm to **Require SSL = none**; do the same for every new realm
Grafana	Log in as `admin`. Kubernetes dashboards are built in; the PostgreSQL dashboard is in the **Extra** folder
Prometheus	**Status → Targets**: the `postgres` job should be UP

4.3 Destroy

Run the job with `ACTION` = `destroy`. It removes the Helm release and the `odoo` and `monitoring` namespaces, including the load balancers. **This permanently deletes the Postgres and Odoo data disks.**

5. Troubleshooting

Every problem below came up during the first setup; each row gives the symptom, the cause and the fix.

Symptom	Cause	Fix
`This command has to be run with superuser privileges`	Setup script started as `ec2-user`	`sudo bash setup-bastion.sh`
`Unable to locate credentials` in "Connect to EKS"	The machine running the job has no AWS credentials	Run the job on the bastion agent (section 3.2); give the bastion an IAM role (section 1.2)
`sudo: kubectl: command not found`	`sudo` does not search `/usr/local/bin`	`sudo ln -sf /usr/local/bin/kubectl /usr/bin/kubectl` (same for `helm`)
`syntax error near unexpected token newline`	A placeholder such as `<your-cluster-name>` was left in the command	Replace it with the real value
Cluster name shows as `****-eks-cluster`	A password equals part of the cluster name, so Jenkins masks it	Use longer passwords
Agent goes offline: "Disk space is below threshold" on `/tmp`	Amazon Linux `/tmp` is a 456 MiB memory folder	Section 3.3, and the JVM tmpdir option in section 3.2 (then Disconnect → Launch agent)
`postgres-0` stuck `Pending`, no events	Its disk (PVC) was not created	`kubectl describe pvc -n odoo`; see next rows
PVC says "Waiting for a volume … ebs.csi.aws.com"	EBS CSI driver missing or broken	Section 1.1

Symptom	Cause	Fix
`ebs-csi-controller` in `CrashLoopBackOff`	Controller cannot get AWS credentials	Pod Identity role `ebs-csi-role` (section 1.1), then `kubectl rollout restart deployment ebs-csi-controller -n kube-system`
Browser cannot open `http://10.0.x.x:300xx`	Node IPs are private; NodePorts are unreachable from outside the VPC	Use the load balancer addresses (services are type `LoadBalancer`)
Load balancer address stays `<pending>`	Public subnets not tagged	Section 1.3; check `kubectl describe svc odoo -n odoo`
Keycloak says "HTTPS required"	Realm requires SSL for external addresses	Realm settings → Require SSL = none (`master` is done by the pipeline)

6. Command cheat sheet

Run these on the bastion as `ec2-user`.

```
# Status
kubectl get pods -n odoo
kubectl get pods -n monitoring
kubectl get pvc -n odoo

# App addresses (EXTERNAL-IP column)
kubectl get svc -n odoo
kubectl get svc -n monitoring

# Logs
kubectl logs -n odoo deploy/odoo --tail=50
kubectl logs -n odoo deploy/keycloak --tail=50
kubectl logs -n odoo postgres-0 -c postgres --tail=50

# Why is a pod stuck?
kubectl describe pod <pod-name> -n odoo | tail -20

# Restart an app
kubectl rollout restart deployment/odoo -n odoo
kubectl rollout restart deployment/keycloak -n odoo
```

```
# EBS driver health
kubectl get pods -n kube-system | grep ebs
kubectl logs -n kube-system deploy/ebs-csi-controller -c ebs-plugin --tail=20
```

Cost note: each AWS load balancer costs roughly $16–18 per month, about $70 per month for all four. Running the job with `destroy` removes them.